

Arrays.java

```

1 // =====***** ARRAYS (Part -- 1) *****===== //
2
3 // Arrays : List Of Elements Of the same type placed in a continous memory location
4
5 // Creating an Array
6 // data Type arrayName [] = new dataType[size];
7
8 import java.util.*;
9
10 public class Arrays {
11     public static void main(String[] args) {
12
13         // ==> Creating an array
14         /*
15         int marks[] = new int[50];
16         int numbers [] = {1,2,3};
17         int moreNumbers [] = {4,5,6};
18         String fruits [] = {"apple","mango","orange"};
19         */
20
21         // ==> Input/Output
22         /*
23         int marks [] = new int[3];
24         Scanner sc = new Scanner(System.in);
25
26         marks[0] = sc.nextInt();
27         marks[1] = sc.nextInt();
28         marks[2] = sc.nextInt();
29
30         System.out.println("Phy : " +marks[0]);
31         System.out.println("che : " +marks[1]);
32         System.out.println("maths : " +marks[2]);
33
34         marks[2] = marks[2] +1;
35         System.out.println("maths : " +marks[2]);
36
37         System.out.println("Length Of Array : " + marks.length);    // For Array Length
38         */
39
40         // Arrays by as Arrgument ==> Array is a by Refrence Passing Arrgument, Bewcause They
         are a changing in other function by refrence a main function
41
42         // ==> Linear Search in Array
43
44         /*
45         int num [] = {2,4,6,8,10,12,14,16,18,20};
46         int key = 16;
47

```

```
48     int index = linear(num, key);
49     if (index == -1){
50         System.out.println("Not Found");
51     }
52     else{
53         System.out.println("Key is at index : " + index);
54     }
55     */
56
57 }
58 // -----> Linear Search Another Function
59 /*
60 public static int linear(int num[], int key) {
61     for(int i=0; i<=num.length; i++){
62         if (num[i] == key){
63             return i;
64         }
65     }
66     return -1;
67 }
68 */
69 }
70
71 // ==> Largest Element Search in Array
72 class Largest_Element{
73     public static int Largest(int num[]){
74         int largest = 0; // Integer.MIN_VALUE = -Infinity
75         // int smallest = Integer.MAX_VALUE = +Infinity
76         for(int i=0; i<num.length; i++){
77             if(largest < num[i]){
78                 largest = num[i];
79             }
80         }
81         return largest;
82     }
83     public static void main(String[] args) {
84         int num [] = {8,5,7,23,45,140};
85         System.out.println("Largest Value is :" + Largest(num));
86     }
87 }
88
89 // ==> Binary Search Search in Array
90 class Binary_Search{
91     public static int Search(int nums[], int key) {
92         int start = 0;
93         int end = nums.length-1;
94         while(start<=end){
95             int mid = start+(end-start)/2; // int mid = start+end/2;
96             if(nums[mid] == key){
97                 return mid;
```

```
98         }
99         else if(nums[mid] < key){
100             start = mid+1;
101         }
102         else{
103             end = mid-1;
104         }
105     }
106     return -1;
107 }
108 public static void main(String[] args) {
109     int nums[] = {4,8,11,54,87,104,120};
110     int key = 120;
111     int result = Search(nums, key);
112     System.out.println(result);
113 }
114 }
115
116 // ==> Reverse an Array
117
118 class Reverse_Array{
119     public static void main(String[] args) {
120         int[] num = {7777,888,99,1,22,333,4444,55555,666666};
121         reverse(num);
122
123         for(int i=0; i<=num.length-1; i++){
124             System.out.print(num[i] + " ");
125         }
126         System.out.println();
127     }
128     public static void reverse(int[] num) {
129         int start = 0;
130         int end = num.length-1;
131
132         while(start<end){
133             // Swap
134             int temp = num[end];
135             num[end] = num[start];
136             num[start] = temp;
137
138             start++;
139             end--;
140         }
141     }
142 }
143
144 // ==> pairs in Array // ***** pairs of n = n(n-1)/2 ***** //
145
146 class Pairs_in_array{
147     public static void main(String[] args) {
```

```
148     int array[] = {1,3,5,7,9};
149     pairs(array);
150 }
151 public static void pairs(int array[]) {
152     int tp = 0;
153     for(int i=0; i<=array.length-1; i++){
154         for(int j=i+1; j<=array.length-1; j++){
155             System.out.print("(" + array[i] + "," + array[j] + ") ");
156             tp++;
157         }
158         System.out.println();
159     }
160     System.out.println("Total Pairs " + tp);
161 }
162 }
163
164 // ==> Subarrays          // ***** n of subarray = n(n+1)/2 ***** //
165 //          : A continous part of array
166
167 class Sub_Arrays{
168     public static void main(String[] args) {
169         int[] nums = {2,4,6,8,10};
170         Subarrays(nums);
171     }
172     public static void Subarrays(int[] nums) {
173         int sa = 0;
174         for(int i=0; i<=nums.length-1; i++){
175             for(int j=i; j<=nums.length-1; j++){
176                 for(int k=i; k<=j; k++){
177                     System.out.print(nums[k] + " ");
178                 }
179                 sa++;
180                 System.out.println();
181             }
182             System.out.println();
183         }
184         System.out.println("Total Sub Arrays is : " + sa);
185     }
186 }
187 }
188
189
190 // =====***** Arrays (Part-1) Complete *****===== //
```